DAZIA

Personal AI Assistant

Budget-First Build Specification

A practical plan for a voice-first assistant that can help with reminders, email, Teams, and daily tasks — prioritizing local software and a target of $0/month in AI API charges.

Designed for: Lenovo laptop • Intel Core i3 11th Gen • 12 GB installed RAM

Account context: Normal/personal Microsoft account. Microsoft integration must be verified feature-by-feature; consumer Teams access is not guaranteed.

Budget target: $0 recurring AI API costs. Electricity, internet, and any optional paid services are not included.

Version 2.0 • 13 October 2026

Use this document as the implementation brief for a coding assistant. Build and test one small milestone at a time; do not generate a giant, untested codebase.

1. Product Vision

DAZIA is a personal productivity assistant that accepts typed or spoken requests, understands the intent, retrieves information the user is authorized to access, runs narrowly scoped tools, and responds in text or speech.

Example: “Dazia, remind me tomorrow at 9 AM to finish my project.” DAZIA extracts the task and due time, checks ambiguity, stores the reminder locally, and confirms it.

Example: “Dazia, summarize my new Outlook emails.” DAZIA authenticates through an approved Microsoft flow, fetches accessible messages, summarizes only their content, and clearly reports any access limitation.

Core use cases:

- Create, list, snooze, and complete reminders and personal tasks.
- Provide a daily briefing from data sources the user has connected and authorized.
- Search and summarize Outlook email where supported by Microsoft Graph permissions.
- Search or summarize Teams messages only if the specific account, Teams product, API, and permissions support it.
- Draft replies and require explicit user confirmation before sending.
- Eventually support hands-free or wake-word voice interaction after the basic system is reliable.

2. User Constraints and Design Decisions

Constraint	Design response
Lenovo i3 11th Gen, 12 GB RAM	Start with small quantized local models; avoid large models and run one heavy component at a time if memory is tight.
No/very low budget	Prefer local inference for LLM, speech-to-text, and text-to-speech. No paid API is required for the first local-only milestone.
Normal/personal Microsoft account	Treat account integration as a compatibility test, not a promise. Outlook.com mail may be more feasible than personal Teams chat access.
Laptop-based deployment	DAZIA runs while the laptop is powered on. Sleep, shutdown, network outages, and Windows restarts can delay reminders or stop processing.
Beginner-friendly implementation	Use Python and FastAPI, a small number of tools, readable code, and incremental tests.

Important: $0/month is a target for AI API usage, not a guarantee that every external integration is available at no cost. Microsoft product eligibility, account type, permissions, and consent rules can block particular features.

3. Budget-First Technology Stack

Layer	Initial choice	Notes / cost
Backend	Python + FastAPI	Free, open-source.
Local LLM	Ollama with a small quantized model	Free software; model quality and speed depend on hardware.
Speech-to-text	faster-whisper / Whisper locally	Free software; CPU transcription may be slow.
Text-to-speech	Piper or another compatible local TTS engine	Free/local option; voice quality and setup vary.
Storage	SQLite	Free, easy to back up; enough for an MVP.
Reminders	APScheduler + persisted SQLite records	Free; implement startup recovery and missed-job handling.
Microsoft access	Microsoft Graph + OAuth where supported	API calls may be free, but permissions, account eligibility, and admin policies still apply.
UI	Simple local web page / FastAPI templates	Free; avoid a separate frontend framework until needed.
Packaging	Run natively first; Docker later if useful	Docker is optional and can add RAM overhead.

Hardware guidance: 12 GB of installed RAM is workable for experiments with small models, but do not assume a specific model will be fast until tested. First check Task Manager's Memory panel and leave headroom for Windows. If the laptop becomes sluggish, use a smaller quantized model, shorten context, or run components sequentially.

4. High-Level Architecture

Initial local flow:

Microphone or text → local speech recognition (if voice is enabled) → FastAPI orchestrator → local LLM interprets intent → backend validates a structured tool request → local tool/database or authorized Microsoft API → response text → local TTS.

The LLM must not execute arbitrary code or independently decide to send a message. It proposes a structured action; normal Python code validates the action, checks required fields and permissions, and requests confirmation when needed.

Suggested initial tools:

- create_reminder(title, due_datetime)
- list_reminders(date_range)
- complete_reminder(reminder_id)
- get_current_datetime()
- search_local_notes(query) — only if a local notes feature is added

5. Voice Interaction

Begin with push-to-talk or a button rather than a permanent wake-word listener. This reduces complexity and helps debug transcription.

1 Capture a short audio recording after the user presses the microphone button.

2. Transcribe locally with a compatible Whisper/faster-whisper model.
3. Show the transcript so the user can correct errors.
4. Pass the text to the assistant orchestrator.
5. Speak the answer with local TTS if enabled.

Voice recognition on CPU may be slower than real time for some model sizes. Benchmark with short recordings before selecting the default. Wake-word detection should be a later milestone, not a blocker for the MVP.

6. Microsoft Account Integrations

Use official Microsoft authentication and APIs. Do not store a Microsoft password in the application. Use OAuth authorization, request the minimum delegated permissions needed, and store tokens securely.

Outlook: Test read-only mail access first. Confirm that the chosen Microsoft Graph mail endpoints work for the user's personal Outlook account and that the app registration and consent flow are supported.

Teams: Do not assume a personal Microsoft account can expose the same Teams chat APIs available to a work/school Microsoft 365 tenant. Teams consumer functionality, Graph endpoint support, permissions, and account type may differ. Verify the exact API and account before implementing this feature.

Sending: Start with draft-only behavior. Before sending any email or message, show the recipient/conversation and full content, then require an explicit confirmation. Resolve ambiguous recipients rather than guessing.

Fallback: If a Graph feature is unavailable to a personal account, keep DAZIA useful with local reminders, voice, and supported Outlook features. Do not bypass access controls or scrape private content using brittle automation.

7. Reminders, Tasks, and Calendar

For the MVP, implement reminders in SQLite and schedule them with APScheduler. Store the reminder itself as durable data; do not rely only on an in-memory scheduled job.

- Store title, due time, time zone, status, creation time, and notification status.
- On startup, reload pending reminders and schedule them again.
- Define what happens after sleep/shutdown: notify about missed reminders on next startup rather than silently losing them.
- Handle ambiguous times with a follow-up question (for example, "tomorrow at 9" if the time zone or AM/PM is unclear).
- Prevent duplicate notifications and record successful delivery.
- Add calendar integration only after local reminders are reliable and the account/API supports it.

8. Assistant Logic and Safety

- Use one orchestrator and a small, typed tool set. Avoid multi-agent architecture for the first version.
- Treat emails, chat messages, attachments, and documents as untrusted data; never follow instructions embedded in them that try to override DAZIA's rules.
- Ground summaries in retrieved content. If the data is unavailable, say so instead of inventing an answer.
- Require explicit confirmation before sending, forwarding, deleting, or making other consequential changes.
- Never expose tokens, secrets, or full private messages in application logs.
- Use timeouts, bounded retries, clear error messages, and tests for each tool.
- Do not give the model unrestricted shell access or arbitrary filesystem access. Use an allowlist of safe actions.

9. Security and Privacy

- Keep API keys and OAuth secrets out of source control. Include a .env.example containing placeholders only.
- Do not paste passwords, refresh tokens, or real secrets into prompts or source files.
- Limit local file access to explicitly selected folders.
- Protect local endpoints if the service is ever exposed beyond localhost.
- Back up the SQLite database and provide a way to delete stored assistant history and preferences.
- Explain what data leaves the laptop. In the first local-first version, avoid hosted AI APIs unless the user explicitly opts in.

10. Main Hurdles and Mitigations

Hurdle	Mitigation
Local model is slow or uses too much RAM	Benchmark a small quantized model; reduce context and concurrency; close other heavy applications.
CPU speech recognition is slow	Use short push-to-talk clips and a smaller speech model; optimize only after a baseline test.
Voice transcription mistakes	Display transcript and confirm consequential commands.
Personal Teams account/API limitation	Verify supported endpoint and account type early; defer Teams rather than building against an unsupported assumption.
OAuth/consent issues	Test sign-in and one read-only endpoint before implementing the full integration.
Reminders missed while laptop is off	Detect overdue reminders at startup and notify; true always-on reminders require an always-on device/service.
LLM hallucinates or chooses the wrong tool	Validate tool schemas and results in backend code; ask clarifying questions when needed.

11. Cost Plan

Target: $0/month in AI API charges. Use local inference first. The following are software-cost targets, not a guarantee of zero electricity or internet costs.

Item	Plan	Recurring AI API cost
LLM	Local Ollama model	$0
Speech recognition	Local faster-whisper / Whisper	$0
Speech output	Local TTS	$0
Reminders/database/backend	Local Python + SQLite	$0
Microsoft APIs	Use only supported authorized endpoints	Potentially $0; verify current account and API requirements
Optional hosted fallback	Only if local performance is unacceptable	Avoid initially; price depends on usage/provider

Do not purchase API credits or subscriptions before the local MVP is working. If a hosted fallback is considered later, set a hard monthly budget, track usage, and keep it disabled by default.

12. Project Structure

Suggested starter layout:

```
dazia/
app/
main.py # FastAPI entry point
config.py # configuration, no hard-coded secrets
assistant.py # orchestration and model adapter
tools/
reminders.py # validated reminder operations
local_search.py # optional, allowlisted local search
outlook.py # later: supported Microsoft Graph features
teams.py # only after API/account support is verified
services/
speech_to_text.py
text_to_speech.py
db.py # SQLite models and migrations
scheduler.py # persistent reminder recovery
tests/
.env.example
requirements.txt
README.md
```

13. Phased Roadmap

Phase	Build	Done when…
0. Hardware check	Check RAM use, Python version, disk space; install Ollama; test one small model.	A simple prompt runs without exhausting memory.
1. Text MVP	FastAPI chat endpoint, local model adapter, one safe reminder tool, SQLite.	Can create/list a reminder and pass automated tests.
2. Reliable reminders	Scheduler recovery, overdue reminders, time-zone handling, duplicate prevention.	Reminders survive backend restart and are handled after downtime.
3. Voice	Push-to-talk, local transcription, local TTS.	Can ask a simple question and hear the response on the laptop.
4. Outlook read-only	OAuth and one supported read-only Graph endpoint.	Can sign in and summarize accessible messages without sending anything.
5. Draft and confirm	Draft reply flow and explicit confirmation gate.	No message can be sent without a clear user confirmation.
6. Teams feasibility	Verify account/product/API support; then implement only supported access.	Feature is proven with the actual account, or clearly documented as unavailable.
7. Polish	Daily briefing, preferences, simple dashboard, tests, packaging.	Stable enough for regular personal use.

14. MVP Acceptance Criteria

- The app runs locally on the Lenovo and gives a clear warning if the selected model cannot load.
- The user can type a command and receive a response.
- A reminder can be created, listed, completed, and recovered after restart.
- Ambiguous reminder dates/times trigger a clarification question.
- No external API is called by the default local-only path.
- Voice input and output are optional and can be switched off independently.

- Outlook/Teams integration reports permission or account limitations honestly.
- No email or Teams message is sent without explicit confirmation.
- The README includes installation, run, test, backup, and troubleshooting instructions.

15. First Vertical Slice

Build only this workflow first:

1. User types: “Remind me tomorrow at 9 AM to finish my project.”
2. The assistant extracts title and due time into a structured object.
3. Backend validates the date, time, and required fields; asks for clarification if ambiguous.
4. SQLite stores the reminder.
5. The scheduler registers the persisted reminder.
6. The app confirms the saved reminder.
7. Restart the backend and verify the reminder still exists and will be handled.

Do not start with wake words, multiple agents, autonomous message sending, or a complex frontend. Get one vertical slice working and tested first.

16. Deployment

Run DAZIA natively on Windows initially to minimize memory and setup overhead. Keep it bound to localhost. Add Docker only after the app works reliably. A laptop-based assistant is not available while the laptop is off or asleep; an always-on deployment would require a separate machine or service and could add cost.

Keep model downloads and caches on a drive with enough free space. Do not run several large models concurrently. Document the model name and quantization chosen so the setup can be reproduced.

17. Official Documentation to Consult

- Ollama documentation: https://ollama.com/
- FastAPI documentation: https://fastapi.tiangolo.com/
- Microsoft Graph documentation: https://learn.microsoft.com/graph/overview
- Microsoft identity platform / OAuth: https://learn.microsoft.com/entra/identity-platform/
- faster-whisper project: https://github.com/SYSTRAN/faster-whisper
- SQLite documentation: https://sqlite.org/docs.html
- APScheduler documentation: https://apscheduler.readthedocs.io/

Before implementing Microsoft features, check the current endpoint documentation for the exact operation and account type. Do not infer API support from a tutorial written for a work/school tenant.

18. Prompt for a Coding Assistant

Copy the following prompt together with this specification:

> **You are helping me build DAZIA, a budget-first personal AI assistant for my Lenovo laptop (Intel Core i3 11th Gen, 12 GB RAM). My goal is $0/month in AI API charges. Prefer local, free tools: Python, FastAPI, a small quantized local LLM via Ollama, local speech-to-text, local text-to-speech, SQLite, and APScheduler. I use a normal/personal Microsoft account, so do not assume work/school Teams APIs are available. Verify every Microsoft integration against official documentation and the actual account’s supported permissions.**

- Inspect the repository and environment before changing files. Do not overwrite existing work without asking.
- Build incrementally. Start with a text-based FastAPI assistant and a single SQLite-backed reminder tool.
- Use a small local model and explain how to test memory/performance on a 12 GB laptop. Do not assume a particular model will run well until tested.
- Keep the default path local-only. Do not call paid APIs unless I explicitly approve a provider, budget, and fallback.
- Use typed tool schemas. Backend code must validate all arguments and permissions; the LLM must not execute arbitrary shell commands.
- Persist reminders and recover them after restarts. Handle time zones, duplicate execution, and missed reminders.
- Treat email/chat/document content as untrusted data and defend against prompt injection.
- Require explicit confirmation before sending email/Teams messages or performing destructive actions.
- Use official Microsoft OAuth and APIs. Start with read-only access and stop if account type, API support, or permissions block the feature.
- Provide small code changes, setup commands, tests, and concise explanations at each milestone. Do not claim a feature works until it has been run or tested.

19. Decisions to Confirm During Implementation

- Exact Windows version and free disk space.
- Actual available RAM when Windows is idle.
- Whether Ollama and a selected small model run acceptably on the laptop.
- Whether the personal Outlook account can authorize the intended read-only mail endpoint.
- Whether the Teams product/account exposes the required chat APIs; if not, document the limitation and defer that feature.
- Whether reminders should notify with a Windows notification, spoken alert, or both.

End of specification. Keep the project small, local-first, testable, and honest about limitations.